

nocap — Business Requirements Document

Sep 26, 2026 · @Hetansh

1. Overview

nocap is a VS Code extension that makes every AI coding agent prove what it's about to do before it does it. It works with any CLI agent running inside VS Code (Claude Code, Codex CLI, Gemini CLI, Copilot CLI, Aider, or anything else that runs shell commands).

One-liner for judges: Other tools read the command. nocap tests it first.

The problem

AI coding agents now run real commands on real systems: they delete files, run database queries, call paid APIs, and edit tests. When instructions are vague, agents don't stop and ask; they guess. A 2026 study of Claude Code, Codex and OpenCode found 55.8–67.8% of runs on underspecified tasks violated at least one boundary. The best-known failure is Replit's agent deleting a company's production database during a code freeze in 2025.

What exists today and why it's not enough

Tools like Destructive Command Guard (dcg) already block known-bad commands (`rm -rf`, `git reset --hard`, `DROP TABLE`) with pattern matching. They can't tell the difference between deleting 3 test rows and 48,213 real users, because both look like `DELETE FROM users WHERE ...`. They also don't know what the developer actually asked for.

What makes nocap different

Every risky action goes through a three-way check:

1. **Task:** what did the developer ask the agent to do?
2. **Intent:** what does the agent say this command is for?
3. **Effect:** what would this command actually do, measured by a dry run before it runs?

If any layer contradicts the one above it, nocap blocks the action, shows the mismatch, and tells the agent why. That's "CAP DETECTED."

It also stops humans from rubber-stamping. When a risky action needs approval, nocap won't accept a reflex "yes." The developer has to type what they expect the action to do, and it only runs if that matches the measured effect.

Demo protections (built deep this weekend)

Protection	The catch it makes on stage
Data destruction	Agent says "removing test users"; dry run shows 48,213 rows including 3 admins
Runaway spend	Agent runs an embedding backfill; nocap estimates about \$340 against a \$20 budget
Cheating the task	Task was "fix the failing test"; agent edited the test's expected value instead of the code

2. Target challenges

Primary target: Assurant "Take Control of AI." The challenge is about privacy, spending, and control over AI tools, and nocap is literally a control layer for AI agents that guards data, money, and secrets. ESPER won Assurant's prize and Best Overall in 2024, and it was a similar "install in one click, protects you in real time" tool.

Challenge	Fit	What we must show to qualify
Assurant: Take Control of AI	Strong	Spend guard, data protection, human stays in control
Best Overall	Strong	Completion, originality, technical depth, practicality
Microsoft: What's Missing?	Strong	Core experience is not a chat window: it's an interceptor with a panel
MLH: Best Use of Gemini API	Strong	Gemini is the judge model for intent and task checks
MLH: MongoDB Atlas	Medium	Audit log, sessions, and verdicts stored in Atlas
MLH: GoDaddy Registry	Easy	Register a domain for the landing page (e.g. a nocap domain)

Challenge	Fit	What we must show to qualify
MLH: Tiger Data	Optional	Only if we add a spend-over-time chart backed by Tiger

Not targeting: Sperry GridLock, Waymo, Blackstone, State Farm, Solana, ElevenLabs, Snowflake, DigitalOcean. None fits without bolting something unrelated on.

Rule of thumb: check each sponsor's exact submission rules on Devpost Friday night. Some require a specific API to be "meaningfully" used, not just mentioned.

3. Scope

We ship one installable VS Code extension that guards any CLI agent, with deep checks for three protections and basic checks for three more.

In scope (must ship)

- VS Code extension, installable from a `.vsix` file, published to the VS Code Marketplace and Open VSX on Sunday morning
- Interception for any CLI agent in the VS Code terminal, through command shims (universal) plus native hooks for Claude Code (deepest integration)
- Task capture: the developer's task is recorded for every agent session
- Deep check 1, data destruction: SQL dry run in a rolled-back transaction, plus file and git impact for `rm` and `git` commands
- Deep check 2, runaway spend: cost estimate for scripts that call LLM APIs, against a budget in `.nocap.yml`
- Deep check 3, cheating the task: diff analysis of test-file edits against the task
- Basic pattern checks for secrets leaking, production systems, and weakened security
- Verdicts: allow, warn, block, or ask the human, with a clear reason sent back to the agent
- Side panel: current task, live action feed, the "CAP DETECTED" block view, allow-once button
- Audit log stored in MongoDB Atlas
- `.nocap.yml` rules file committed to the repo

Out of scope (do not build)

- The "project health" GitHub and docs watcher (a different product)
- Blocking commands typed by humans (nocap guards agents; humans are only guarded if a stretch goal is reached)
- Windows support (macOS and Linux only; state this in the README)
- Deep checks for the other three categories
- Stopping a malicious agent that actively tries to evade nocap. Like dcg, we assume the agent is well-meaning but fallible, and we say so.
- User accounts, login, or a hosted cloud service

Success criteria for Sunday

1. All three demo scenarios run end to end, live, three times in a row without a failure
2. Safe commands pass through in under 50 ms, so nocap never feels slow
3. A deep check returns a verdict in under 5 seconds
4. A new person can install the `.vsix` and protect a project in under 2 minutes
5. Public GitHub repo with a README, a demo GIF, and install steps

4. How it works

A VS Code extension can see terminal commands but can't stop them, so nocap blocks in two other places: command shims (works with any agent) and native agent hooks (richer for Claude Code). The extension installs both, runs the local server, and shows everything in a side panel.

```

flowchart LR
    A[Any CLI agent in VS Code terminal] -->|runs psql, rm, git, python...| S[nocap shims on PATH]
    S --> CC[Claude Code]
    CC -->|PreToolUse + UserPromptSubmit hooks| H[nocap hook script]
    H --> S
    S --> D[nocap daemon localhost:7777]
    D --> H
    D --> F{Fast check}
    F -->|safe| OK[allow under 50 ms]
    F -->|risky| M[Measurers: SQL dry run, file and git impact, cost, diff]
    M --> J[Gemini judge: task vs intent vs effect]
    J --> P[Policy: .nocap.yml budget and rules]
    P --> V[Verdict]
    V --> S
    V --> H
    V --> UI[VS Code side panel]
    V --> DB[(MongoDB Atlas audit log)]

```

The four parts

1. **Shims (universal interception).** Small wrapper scripts named `psql`, `sqlite3`, `rm`, `git`, `python`, `node`, `curl`, `terraform`, `kubectl`. The extension puts their folder first on PATH, but only inside VS Code terminals, using VS Code's `environmentVariableCollection` API. Any agent that runs one of these commands hits the shim first. The shim asks the daemon for a verdict, then either runs the real command or exits with an error the agent can read.
2. **Native hooks (Claude Code).** Claude Code's `PreToolUse` hook sees every shell command and file edit before it happens, including the agent's own description of the command. Its `UserPromptSubmit` hook gives us the developer's task. This is our richest integration and the one we demo.
3. **The daemon.** A local Node server on `localhost:7777` that runs the checks, calls the judge model, applies the rules, logs to MongoDB, and pushes live events to the panel over a WebSocket.
4. **The extension.** Starts the daemon, installs shims and hooks, and shows the panel.

How nocap forces the agent to explain itself

This is the core mechanic, and it works with any agent that can read error output:

1. The agent runs a risky command with no stated intent, e.g. `psql -c "DELETE FROM users WHERE ..."`.
2. The shim blocks it and prints: `nocap: this command changes data. Re-run it as NOCAP_INTENT="<why you are running it>" psql ...`
3. The agent reads that, re-runs with an intent, and now all three layers can be checked.
4. With Claude Code, the intent comes for free from the hook, since the agent already describes each command.

How the task is captured

- Claude Code: automatically, from the `UserPromptSubmit` hook
- Any other agent: the developer types it once in the panel's "Current task" box, or runs `nocap task "fix the failing login test"`
- No task set: risky actions become "ask the human" instead of being judged automatically

What happens on each command

1. **Fast check (under 50 ms, no AI):** a pattern classifier decides if the command is risky and which category it belongs to. Safe commands (`ls`, `npm test`, `git status`) pass straight through.
2. **Measure:** the matching measurer computes the real effect (rows, files, dollars, diff facts).
3. **Judge:** Gemini gets the task, the intent, and the measured facts, and returns structured JSON: does the intent fit the task, and does the effect match the intent?
4. **Policy:** hard rules from `.nocap.yml` override everything (budget caps, protected tables, protected branches).
5. **Verdict:** allow, warn, block, or ask the human, with a one-line reason written for the agent to act on.

Known limits (say these out loud to judges)

- An agent calling a binary by full path (`/bin/rm`) skips the shim. Claude Code hooks still catch it; for other agents it's a documented gap.
- Dry runs on a live database are safe for data but can briefly take locks. Real deployments should dry-run against a replica. For the demo we use a local Postgres with a statement timeout.

- Database triggers that call outside services would still fire during a dry run. We document this.

Tech stack

Layer	Choice	Why
Extension	TypeScript, VS Code Extension API, webview panel	Required by VS Code
Daemon	Node 20, TypeScript, Fastify, <code>ws</code>	Same language as the extension; shared types
Shims and hooks	POSIX <code>sh</code> scripts calling the daemon with <code>curl</code>	Start in milliseconds, no runtime needed
Judge model	Gemini API (<code>@google/genai</code>), JSON output mode	Fast, cheap, counts for the Gemini prize
Demo database	Postgres 16 in Docker	Real transactions, and even <code>DROP TABLE</code> can be rolled back
Audit log	MongoDB Atlas free tier	Flexible event documents, MongoDB prize
Git and diffs	<code>simple-git</code> , plain <code>git diff</code>	Reliable and quick

5. Functional requirements

Each protection has a trigger, a measurement, a judgment, and a verdict, and each has a pass/fail test we run before the demo. Requirement IDs (FR-D1 and so on) are referenced in the task lists in sections 7–9.

5.1 Data destruction (deep)

Trigger. SQL run through `psql` , `sqlite3` , or a Claude Code Bash call, where the statement is `DELETE` , `UPDATE` , `DROP` , `TRUNCATE` , or `ALTER ... DROP` . Also `rm` (recursive or globbed), `git reset --hard` , `git clean -fd` , `git checkout -- <path>` , `git push --force` , `git branch -D` .

Measurement.

- **FR-D1 SQL dry run:** open a connection from `.nocap.yml` (or `DATABASE_URL`), `BEGIN`, set a 3 s statement timeout and 1 s lock timeout, run the statement, record the row count, then `ROLLBACK`. Never commit.
- **FR-D2 Cascades:** inside the same transaction, read `pg_stat_xact_user_tables` to count rows deleted or updated in every table, which catches foreign-key cascades the agent didn't mention.
- **FR-D3 Sensitive rows:** count affected rows matching rules in `.nocap.yml` (e.g. `users.role = 'admin'`, any row in a protected table).
- **FR-D4 Sample:** capture up to 10 affected rows, with columns marked sensitive (email, name, password hash) removed, so the judge can see what kind of rows these are.
- **FR-D5 Files:** for `rm`, expand globs, count files and bytes (stop walking at 10,000 files), list files with uncommitted or untracked changes from `git status --porcelain`, and flag any target outside the workspace.
- **FR-D6 Git:** for `reset --hard`, `clean`, and `checkout --`, count uncommitted changes that would be lost. For force-push, count remote commits that would be overwritten (`git rev-list local..remote`).

Judgment. Gemini receives task, intent, and the facts above. It returns whether the scope of the effect matches the intent ("test users" vs 48,213 rows of real-looking users).

Verdict rules. Any target outside the workspace → block. Any sensitive row or protected table hit and not named in the intent → block. Row count or file count much larger than the intent implies → block. Otherwise → allow, logged.

Acceptance test. Demo DB has 48,391 users: 178 test users (`@test.local`), 3 admins, the rest real-looking. Task: "clean up the test users." Agent runs `DELETE FROM users WHERE last_login_at < now() - interval '90 days'`. nocap shows 48,213 rows, 3 admins, 2 cascaded tables, and blocks within 5 s. The agent retries with `WHERE email LIKE '%@test.local'`, nocap measures 178 rows, and allows it.

5.2 Runaway spend (deep)

Trigger. `python`, `node`, `tsx`, or `npx` running a script file whose code, or a local file it imports (one level deep), uses an LLM or embeddings SDK or API URL (OpenAI, Anthropic, Gemini, Cohere, Voyage).

Measurement (the AI reads, math decides).

- **FR-S1 Extraction:** Gemini reads the script and returns JSON: provider, model, calls per item, max output tokens, and where the loop's items come from (SQL query, file, list, `range(n)`, or unknown).
- **FR-S2 Loop size:** nocap resolves the real count deterministically: runs the SQL as a read-only `SELECT count(*)`, counts lines in the file, or reads the range.
- **FR-S3 Tokens per call:** sample up to 20 real items and estimate tokens (`characters ÷ 4` for the MVP).
- **FR-S4 Price:** a `prices.json` in the repo with dollars per million input and output tokens per model, with the date it was checked.
- **FR-S5 Estimate:** items × calls per item × token cost, shown as a low-high range with a confidence label. Unknown loop source → "can't estimate" and ask the human.

Verdict rules. Estimate above `budget.per_command_usd` (default \$5) → block. The session total above `budget.per_session_usd` (default \$20) → block. The message to the agent suggests a safe next step ("run it on 100 items first with a LIMIT").

Acceptance test. Task: "add embeddings for new documents." Agent runs `python scripts/backfill_embeddings.py`, which loops over every document. nocap estimates about \$340 (seeded to land there) and blocks. Agent adds `WHERE embedding IS NULL`, the estimate drops under \$1, and nocap allows it.

5.3 Cheating the task (deep)

Trigger. An edit to a test file (globs in `.nocap.yml`, default `**/*.test.*`, `**/*.spec.*`, `**/test_*.py`, `tests/**`). Caught before it happens through Claude Code's `Edit / Write / MultiEdit` hook, and through the `git commit` shim for all other agents.

Measurement (deterministic diff facts).

- **FR-T1** Expected values changed in assertions (`expect(x).toBe(15)` → `toBe(10)`)
- **FR-T2** Tests deleted, or skip markers added (`.skip`, `xit`, `@pytest.mark.skip`, `xfail`)
- **FR-T3** Assertion count went down
- **FR-T4** Whether any non-test source file changed in the same session
- **FR-T5** Snapshot files overwritten, or test timeouts raised

Judgment. Gemini gets the task, the diff, and the facts, and answers: was the agent asked to change the tests, or to make the code satisfy them?

Verdict rules. Task asks to fix code, and the edit weakens a test with no matching source fix → block. The message tells the agent what to do instead: "The task is to fix the code. The test expects a 15% discount and `applyDiscount` returns 10%. Fix `applyDiscount`."

Acceptance test. Demo repo has a bug in `cart.ts` and a failing test in `cart.test.ts`. Task: "fix the failing cart test." Agent edits the expected value in the test. nocap blocks it with the message above, the agent fixes `cart.ts`, and the test passes.

5.4 Basic checks (pattern only)

ID	Category	Catches	Verdict
FR-B1	Secrets	<code>cat .env</code> , printing env vars, <code>git add .env</code> , key patterns (<code>sk-</code> , <code>AKIA</code> , <code>ghp_</code> , private key headers), <code>curl</code> sending a file to a domain not on the allowlist	Block
FR-B2	Production	Hosts or DB URLs listed as prod in <code>.nocap.yml</code> , <code>kubectl</code> on a prod context, <code>terraform apply</code> or <code>destroy</code> , deploy scripts	Ask human
FR-B3	Security	<code>chmod 777</code> , <code>curl ... sh</code> , turning off TLS checks, disabling auth middleware	Warn

5.5 Rules that apply to everything

- **FR-G1** Safe commands get a verdict in under 50 ms with no AI call.
- **FR-G2** Every risky action is logged: task, intent, command, measured facts, verdict, reason, and time taken.
- **FR-G3** Overriding a block goes through the human intent check (section 5.6). A matching answer unlocks that exact command (by hash) for 60 seconds, and is logged.
- **FR-G4** If the daemon is down: safe commands pass, risky ones are blocked with "nocap is offline."
- **FR-G5** If Gemini times out (over 4 s) or errors: fall back to the facts and rules alone, and mark the verdict "rules only."
- **FR-G6** Every block message is written for the agent: what was wrong, and what to do instead.
- **FR-G7** Nothing leaves the machine except the Gemini call, and that call never includes sensitive columns or secrets.

5.6 Rubber-stamp guard: the human has to say why

Why. Developers approve agent prompts dozens of times an hour and stop reading them. A "yes" clicked in half a second isn't real approval, so nocap asks the human the same question it asks the agent: what do you expect this to do?

Trigger. Every action the pipeline marks risky (`human_check: risky` , the default), only blocked ones (`blocked_only`), or never (`off`). It applies whatever approval mode the agent is in, including auto-accept (confirm this in A0.2). Safe commands never trigger it.

- **FR-H1 Hold:** the agent's command waits while the human answers. The hook or shim keeps the request open for up to 5 minutes; no answer means deny, with the reason "the developer didn't confirm."
- **FR-H2 Ask before revealing:** a pop-up in VS Code shows the command and asks "What do you expect this to do?" It does *not* show nocap's measured numbers yet, so the human can't just skim and agree.
- **FR-H3 Reject reflex answers:** answers under 4 words, answers like "yes," "ok," "y," or "go," and answers sent within 2 seconds of the pop-up opening are refused with "Tell nocap what you expect this to do."
- **FR-H4 Compare:** the judge checks the human's expectation against the measured effect and the task, exactly as it does for the agent.
- **FR-H5 Verdict:** a match runs the action, and nocap tells Claude Code "allow" so the human isn't asked twice. A mismatch shows the human a CAP DETECTED screen ("You expected: remove a few test accounts. This actually deletes 48,213 users, including 3 admins"). To run it anyway, they must type the key number (`48213`), like GitHub's "type the repo name to delete."
- **FR-H6 Log:** the answer, how long it took, and the outcome go to the audit log. The panel counts "blind approvals prevented."

Acceptance test. The agent's narrowed `DELETE` (178 rows) triggers the pop-up. Typing "ok" is refused. Typing "delete the QA test accounts" matches and runs. Running the broad version with the same answer shows a mismatch and requires typing `48213` .

Friction rule. If it prompts more than a few times an hour on normal work, the risky list is too broad. Tune the classifier, don't train people to ignore the pop-up.

6. Shared contracts

Lock these in the first hour; after that, all three people build against them without waiting on each other. Put the TypeScript types in one shared file, `packages/shared/types.ts`, that the extension and the daemon both import. Changes after hour 2 need all three people to agree.

6.1 Daemon API (localhost:7777)

Endpoint	Who calls it	Purpose
<code>POST /v1/check</code>	Shims, Claude Code hook	Ask for a verdict on one action (stays open while a human check is pending)
<code>POST /v1/task</code>	Claude Code hook, panel	Set the current task for a session
<code>POST /v1/human-intent</code>	Extension pop-up	The human's typed expectation (or the typed confirm number) for a pending check
<code>GET /v1/health</code>	Extension, shims	Is the daemon up?
<code>WS /v1/stream</code>	Panel, extension	Live events: <code>check.started</code> , <code>check.finished</code> , <code>task.updated</code> , <code>human.needed</code> , <code>human.answered</code>

6.2 Check request

```
{
  "session_id": "vscode-term-3",
  "source": "shim | claude_hook | commit_shim",
  "agent": "claude-code | codex | gemini-cli | unknown",
  "cwd": "/Users/dev/shop-app",
  "tool": "bash | edit | write",
  "command": "psql $DATABASE_URL -c \"DELETE FROM users WHERE ...\"",
  "edit": { "file": "src/cart.test.ts", "old": "...", "new": "..." },
  "intent": "Removing the test users created during QA"
}
```

`edit` is only present for file edits. `intent` is null when the agent gave none.

6.3 Verdict response

```
{
  "check_id": "chk_01J...",
  "verdict": "allow | warn | block | ask",
  "category": "data | spend | test_cheat | secrets | prod | security | safe",
  "headline": "CAP DETECTED: 48,213 rows, not 'test users'",
  "facts": [
    { "label": "Rows deleted", "value": "48,213", "severity": "high" },
    { "label": "Admin accounts hit", "value": "3", "severity": "high" },
    { "label": "Cascaded tables", "value": "orders, sessions", "severity":
"medium" }
  ],
  "layers": {
    "task_fit": { "ok": true, "why": "Cleaning test users matches the task"
},
    "intent_effect": { "ok": false, "why": "Intent says test users; 99.6% of
affected rows are not test accounts" }
  },
  "reason_for_agent": "Blocked by nocap: this deletes 48,213 users including
3 admins. Test users have emails ending in @test.local. Narrow the WHERE
clause.",
  "mode": "full | rules_only",
  "latency_ms": 2140
}
```

6.4 How each caller shows the verdict

Caller	Allow	Block or ask
Shim	Runs the real binary, exit code from it	Prints <code>reason_for_agent</code> to stderr, exits with code 86
Claude Code hook	Exit 0	Returns <code>permissionDecision: deny</code> (or <code>ask</code>) with <code>reason_for_agent</code> as the reason
Commit shim	Runs <code>git commit</code>	Prints reason, exits 86, commit doesn't happen

6.5 Plug-in interfaces inside the daemon

```
interface Measurer {
  category: Category;
  matches(req: CheckRequest): boolean;    // cheap, no I/O
  measure(req: CheckRequest, ctx: Ctx): Promise<Measurement>;
}

interface Measurement {
  facts: Fact[];                        // shown in the panel
  judgeContext: object;                 // what the judge sees (no secrets)
  hardBlock?: string;                   // rule-based block reason, skips the judge
}

interface Judge {
  judge(input: {
    task: string | null; intent: string | null;
    category: Category; judgeContext: object;
  }): Promise<JudgeResult>;    // task_fit, intent_effect, reason_for_agent
}
```

Adding a new category later means writing one `Measurer`. Nothing else changes.

6.6 .nocap.yml (lives in the repo)

```
version: 1
database:
  url_env: DATABASE_URL
  protected_tables: [payments, audit_log]
  sensitive_rows:
    - table: users
      where: "role = 'admin'"
  redact_columns: [email, name, password_hash, phone]
human_check: risky # risky | blocked_only | off
budget:
  per_command_usd: 5
  per_session_usd: 20
tests:
  globs: ["**/*.test.*", "**/*.spec.*", "**/test_*.py", "tests/**"]
production:
  hosts: ["db.prod.internal"]
  kube_contexts: ["prod"]
network:
  allow_domains: ["api.github.com", "registry.npmjs.org"]
```

6.7 MongoDB Atlas collections

Collection	One document per	Key fields
sessions	Agent session	session_id, agent, task, repo, started_at, spend_usd
checks	Risky action checked	full request, verdict response, created_at
human_checks	Human intent pop-up	check_id, answer, seconds_to_answer, rejected_attempts, outcome

7. Role A: AI Connector and extension

Owns everything that touches an AI agent, plus the VS Code extension shell and the human intent pop-up. Two people now write all the code, so every estimate assumes you build with an AI coding agent (and run nocap on your own build from Saturday on: that's a great story for judges). About 30 hours.

Phase 0: setup (hours 0–2)

- ☐ **A0.1 Contracts (1 h, with B; C listens in).** Agree on section 6. *Done when:* `types.ts` is merged.
- ☐ **A0.2 Confirm Claude Code hook behavior (0.5 h).** Save real hook payloads for a Bash call and an Edit call to `fixtures/hooks/`. Also confirm two things: hooks still fire in auto-accept mode, and the hook `timeout` can be raised to 300 s (needed for the pop-up). *Done when:* both are confirmed or a workaround is written down.
- ☐ **A0.3 Gemini access (0.5 h).** One JSON-mode call with a schema works.

Phase 1: interception (hours 2–9)

- ☐ **A1 Claude Code PreToolUse hook (3 h).** `nocap-hook` maps the hook JSON to a `CheckRequest` (Bash command + the agent's description as intent; Edit/Write/MultiEdit as `edit`), waits for the verdict (up to 300 s if a human check is pending), and returns allow, deny, or ask. When the human already confirmed in nocap's pop-up, return allow so Claude Code doesn't ask again. *Done when:* a saved fixture gets a deny from B's stub, and Claude Code shows the reason.
- ☐ **A2 Task capture hook (1 h).** `UserPromptSubmit` posts the task to `/v1/task`. *Done when:* the task shows up in the panel.
- ☐ **A3 Hook installer (1.5 h).** Merge nocap's hooks into `.claude/settings.json` without overwriting (back up first, no duplicates), and check they're still there on startup. *Done when:* running it twice leaves one copy of each hook.
- ☐ **A4 Shims, first set (1.5 h).** `psql`, `rm`, `git`, `python` only. More binaries later if there's time. *Done when:* `ls` works normally and `rm -rf ./src` is blocked in a shimmed terminal.
- ☐ **A5 Intent protocol (0.5 h).** The "re-run with `NOCAP_INTENT`" message, tested with Claude Code and one other agent. *Done when:* the other agent retries with an intent on its own.

Phase 2: the extension shell (hours 6–14, taken over from the old Role C)

- ☐ **A6 Scaffold and packaging (1 h).** TypeScript extension, `esbuild`, `vsce` package to a `.vsix`. *Done when:* installing the `.vsix` in a clean VS Code shows nocap in the activity bar.
- ☐ **A7 Daemon lifecycle and status bar (2 h).** Start the bundled daemon on workspace open, restart on crash (max 3), stop on close. Status bar: "nocap: on / off / offline." *Done when:* killing the daemon brings it back within 2 s.

- ☐ **A8 PATH injection (1 h).** `environmentVariableCollection.prepend('PATH', ...)` plus a "Relaunch terminals" button. *Done when:* `which rm` points to the shim in a new terminal.
- ☐ **A9 Commands (1 h).** Enable in this workspace (installs hooks, shims, and `.nocap.yml`), Disable, Set task, Open panel. *Done when:* each works from the command palette.
- ☐ **A10 Human intent pop-up (3 h, FR-H1 to H5).** On `human.needed`, open a modal webview built from C's design: the command, a text box, and a submit button. Enforce the reflex rules (under 4 words, "yes/ok/go," or under 2 seconds → refused). After the verdict, show the match or the mismatch screen with the type-the-number confirm. *Done when:* the section 5.6 acceptance test passes.
- ☐ **A11 Block notifications (0.5 h).** A VS Code warning toast with a View button on every block.

Phase 3: the judge (hours 12–24)

- ☐ **A12 Judge module (2 h).** One Gemini call returns `task_fit`, `intent_effect`, a suggested verdict, and `reason_for_agent`, as schema-checked JSON. Works for agent intent and human intent. *Done when:* B's pipeline gets typed results.
- ☐ **A13 Data prompt (1.5 h).** *Done when:* 48,213 rows blocks, 178 rows allows.
- ☐ **A14 Spend prompts (2.5 h).** The extractor (FR-S1: provider, model, calls per item, loop source, or "unknown") and the fit check. *Done when:* the \$340 case blocks and a script with no loop says unknown.
- ☐ **A15 Test-cheat prompt (2 h).** *Done when:* the `cart.test.ts` edit blocks with the `applyDiscount` hint, and "update the test to the new pricing" is allowed.
- ☐ **A16 Human intent prompt (1 h).** Compares the human's expectation with the effect. *Done when:* "delete the QA test accounts" matches 178 rows and mismatches 48,213.
- ☐ **A17 Redaction and fallback (1.5 h).** Strip redacted columns and secrets before every call; 4 s timeout, one retry, then "rules only." *Done when:* a unit test shows no email reaches Gemini, and turning off WiFi still gives a verdict.
- ☐ **A18 Eval set (2.5 h).** 30 labeled cases (7 each for data, spend, tests, and human intent, plus 2 safe), `npm run eval` prints the score. Target 27 of 30 with no false blocks on safe cases. *Done when:* the score is in the README.

Phase 4: real agents (hours 24–30)

- ☐ **A19 Live tuning (3 h).** Run every demo scenario 5 times with Claude Code and tune messages until the agent fixes itself on the first retry each time.

- ☐ **A20 Message style guide (0.5 h).** What was wrong (one number), why it doesn't match, what to do instead. C writes the final wording; A makes sure the agent responds to it.

Cut or moved to stretch: the commit shim for non-Claude agents, the `nocap` CLI, verdict caching, and other agents' native hooks.

Handoffs. Needs B's stub daemon by hour 3 and measurer outputs by hour 14. Needs C's pop-up and notification designs by hour 8. Gives B the judge module by hour 14.

8. Role B: Backend and panel

Owns the daemon, the measurers, the human-check flow, and building the side panel from C's designs. About 32 hours, still the heaviest role, so B gets first claim on the cut list in section 10.

Phase 0: foundation (hours 0–3)

- ☐ **B0.1 Contracts (1 h, with A).**
- ☐ **B0.2 Monorepo (1 h).** pnpm workspaces: `shared`, `daemon`, `extension`, `shims`. *Done when:* A can clone and build.
- ☐ **B0.3 Services (0.5 h).** Postgres 16 in Docker, a MongoDB Atlas free cluster. *Done when:* both connect.
- ☐ **B1 Stub daemon (1 h).** Every route from section 6.1 with hard-coded verdicts. **Ship by hour 3; it unblocks A.**

Phase 1: the pipeline (hours 3–9)

- ☐ **B2 WebSocket stream (1 h).** All events from section 6.1.
- ☐ **B3 Fast classifier (3 h).** Parse with `shell-quote`, split `&&`, `;`, `|`, look inside `bash -c`, `$(...)`, and heredocs. *Done when:* 60 test commands classify correctly, safe ones in under 5 ms.
- ☐ **B4 Config loader (1 h).** `.nocap.yml` with `zod`, defaults, and reload on change.
- ☐ **B5 Pipeline (2 h).** `classify` → `measurer` → `judge` → `policy` → `verdict`, with timings; any error becomes "rules only." Keep tasks per session in memory, written through to MongoDB.

Phase 2: measurers (hours 8–20)

- ☐ **B6 SQL extraction (1.5 h).** From `psql -c`, `-f`, and heredocs; parse with `node-sql-parser`; refuse `COMMIT` and multiple statements.
- ☐ **B7 Dry run (2 h, FR-D1).** `BEGIN`, 3 s statement and 1 s lock timeouts, run, count, `ROLLBACK` in `finally`. *Done when:* 100 dry-run deletes leave the table unchanged.
- ☐ **B8 Cascades, sensitive rows, sample (3 h, FR-D2 to D4).** `pg_stat_xact_user_tables` before and after; count rule-matching rows before and after; `RETURNING *` sample of 10 with redacted columns removed. *Done when:* the demo reports 48,213 rows, 3 admins, and 2 cascaded tables.
- ☐ **B9 File measurer (2 h, FR-D5).** `rm` targets, globs, file counts, unsaved git files, outside-workspace hard block. Git commands stay pattern-only (the deep git measurer is cut).
- ☐ **B10 Spend measurer (3 h, FR-S2 to S5).** SDK detection, loop count via a read-only count query, a 20-item token sample, `prices.json`, and the low-high estimate. *Done when:* about \$340 for the broad script, under \$1 for the fixed one.
- ☐ **B11 Test diff facts (2.5 h, FR-T1 to T5).** Changed expected values, deleted or skipped tests, fewer assertions, and whether source files changed. Jest/Vitest first; pytest only if time allows.
- ☐ **B12 Basic pattern rules (1 h, FR-B1 to B3).** Rules as data, not code.

Phase 3: rules and the human check (hours 18–24)

- ☐ **B13 Policy engine (1.5 h).** Hard rules beat the judge: outside workspace, protected tables, budget, prod.
- ☐ **B14 Human check flow (2.5 h, FR-H1, H4 to H6).** When a human check is needed, hold the `/v1/check` request open (up to 300 s), emit `human.needed`, accept `/v1/human-intent`, send the answer to A's judge, then release the verdict. Also verifies the typed confirm number on mismatch. *Done when:* the section 5.6 acceptance test passes end to end.
- ☐ **B15 Audit log (1 h).** MongoDB writes that never slow a verdict, with a local `.jsonl` fallback.
- ☐ **B16 Latency check (0.5 h).** Safe p50 under 50 ms, deep checks under 5 s, numbers in the README.

Phase 4: the panel, built from C's designs (hours 14–26)

- ☐ **B17 Panel webview (1.5 h).** A `WebviewView` in plain TypeScript, using VS Code's CSS variables so it matches every theme. Connects to the WebSocket and reconnects.
- ☐ **B18 Task card (1 h).** Current task, its source, an edit button, and the "no task set" banner.
- ☐ **B19 Live feed (2 h).** One row per action; safe commands collapse into a counter.
- ☐ **B20 CAP DETECTED view (3 h).** Task / Agent says / Actually does columns, facts, the message sent to the agent, and buttons. Match C's mockup closely; this is the screenshot everyone will share.

Phase 5: demo fixtures (hours 20–26)

- ☐ **B21 Seed data (2 h).** Exact numbers from section 5: 48,391 users (178 test, 3 admins), cascading `orders` and `sessions`, and a `documents` table sized for about \$340.
- ☐ **B22 Demo repo and reset (1.5 h).** The `shop-app` repo with the `cart.ts` bug and failing test, the backfill script, and `pnpm demo:reset` (under 10 s).

Cut or moved to stretch: the deep git measurer, pytest support, the spend meter, the stats header, and the history tab.

Handoffs. Gives A the stub daemon by hour 3 and measurer outputs by hour 14. Needs A's judge by hour 14. Needs C's panel mockups by hour 12, and asks C for visual review of the built panel by hour 24.

9. Role C: Design and launch (no coding)

Owns how nocap looks, reads, and gets presented: every screen, every message, the brand, the demo video, and the pitch. No code at all. A and B build from C's mockups, so C's first deadline (hour 8 for the pop-up, hour 12 for the panel) is what unblocks them. About 26 hours.

Phase 0: learn the ground (hours 0–2)

- ☐ **C0.1 Sit in on the contracts session (1 h).** Listen for what data each screen can show (the `facts`, `layers`, `headline`, and `reason_for_agent` fields in section 6.3). Design only with data that exists.

- ❑ **C0.2 Study the setting (1 h).** Open VS Code in light and dark themes, install Claude Code, and screenshot its side panels and pop-ups. nocap has to look like it belongs in VS Code. Grab the free Codicons icon set and the VS Code Figma toolkit (search "VS Code UI kit" in the Figma community).

Phase 1: design the screens (hours 2–12)

- ❑ **C1 Brand (1.5 h).** Check that "nocap" is free on the VS Code Marketplace and npm (just search both sites). Logo, an emoji-style icon, one accent red for CAP DETECTED, a green for allowed, and a tagline. *Done when:* the team agrees in a 5-minute vote.
- ❑ **C2 Human intent pop-up (2 h, due hour 8).** Four states: asking (command + "What do you expect this to do?"), reflex answer refused ("ok" rejected), match (green, running), and mismatch ("You expected... this actually..." + "type 48213 to run anyway"). *Done when:* A has the mockups and can build from them.
- ❑ **C3 CAP DETECTED screen (3 h, most important).** Three columns: Task / Agent says / Actually does, each with a check or cross. Big numbers for high-severity facts. The message sent to the agent. Buttons. Light and dark versions. *Done when:* a stranger explains what happened within 5 seconds (see C7).
- ❑ **C4 Side panel (2.5 h, due hour 12).** Task card (with task, empty, and "no task set" states), live feed rows (allowed, warned, blocked, collapsed safe counter), and the offline state. *Done when:* B has the mockups.
- ❑ **C5 Status bar and notification (0.5 h).** The three status bar states and the block toast.
- ❑ **C6 Handoff notes (1 h).** For each screen: sizes, spacing, and which VS Code color to use (ask B for the list of VS Code theme color names, and label colors with those names instead of hex codes, so dark mode works automatically).

Phase 2: words and testing (hours 10–20)

- ❑ **C7 Five-second tests (2 h).** Show the CAP DETECTED and pop-up mockups to 5 strangers at the event for 5 seconds each, then ask "what just happened?" Fix whatever confuses 2 or more people.
- ❑ **C8 All the wording (2 h).** Write every piece of text: block headlines, pop-up text, refused-answer text, notifications, empty states. Keep a single "strings" doc so A and B copy text from it and don't make it up. Coordinate with A on messages the agent reads (A20).
- ❑ **C9 Review the build (1 h, hour 24).** Compare B's panel and A's pop-up to the mockups, and give a list of fixes with screenshots.

Phase 3: launch (hours 20–34)

- ☐ **C10 Landing page (2 h).** Built in a no-code tool (Framer or Carrd): tagline, demo GIF, three-step explainer, install button. Connect a domain bought through GoDaddy Registry (that's the MLH GoDaddy prize).
- ☐ **C11 Demo video (2 h, Saturday night).** Screen-record a perfect run of all scenarios with the team, trim it to about 2 minutes, add captions. Save it on two laptops. This is also the backup if the live demo fails.
- ☐ **C12 README visuals (1 h).** The CAP DETECTED GIF and the architecture diagram as a clean image. A or B writes the technical README text; C makes it look good.
- ☐ **C13 Devpost page (1.5 h).** Write-up (problem, how it works, challenges), gallery images, the video, and a sentence for each prize you enter.
- ☐ **C14 Demo runbook (1 h, with A).** The exact prompts for each scenario, the reset step, and what to do if something breaks, so anyone can run it.
- ☐ **C15 Pitch (2 h, whole team rehearses).** C is the speaker; A drives the laptop. Learn the section 11 script and the judge answers in section 13. *Done when:* three clean run-throughs.

Handoffs. Gives A the pop-up and notification designs by hour 8. Gives B the panel designs by hour 12. Reviews the build at hour 24.

10. Timeline and milestones

The one date that matters: the full data-destruction scenario works end to end by hour 18 (Saturday morning). Everything else builds on it. Hours count from when hacking starts; adjust to the official schedule.

Hour	Milestone	Gate: must be true to move on
0–2	Contracts locked, monorepo builds	<code>types.ts</code> merged; everyone builds locally
3	Stub daemon live	A's hook and C's panel both talk to it
10	Interception works	Claude Code and a shim both get blocked by the stub; panel shows events live
14	Real SQL dry run	Real row counts and cascades appear in the panel

Hour	Milestone	Gate: must be true to move on
18	Scenario 1 end to end	Claude Code tries the bad delete, gets blocked with real numbers, fixes its query, gets allowed
24	Scenarios 2 and 3 end to end	Spend and test-cheat both block and recover with a real agent
26	Feature freeze	No new features after this. Only fixes, polish, and demo prep
28	Backup video recorded	Saved on two laptops
30	Eval score and latency numbers	Both in the README
32	Three clean rehearsals	Full demo, three times in a row, no failures
34	Submitted	Devpost, repo public, landing page live

Sleep plan: A and B stagger: B sleeps hours 11–15, A sleeps 15–19, both awake for the hour-18 gate or right after. C should be awake and delivering by hours 8 and 12 (the design handoffs), then can sleep hours 13–19 and take the late-night video and launch shift.

Publishing (hour 30, A, about 1 hour): create a publisher (a free Azure DevOps organization and an access token), run `vsce publish`, then publish the same `.vsix` to Open VSX so Cursor and Windsurf users can install it too. Do this after the feature freeze so a publishing hiccup never eats build time.

Two coders is the biggest risk in this plan. A and B are each carrying about 30 hours in a 36-hour event. Use AI coding agents heavily, and cut in this order the moment you fall behind a gate:

1. Spend becomes a basic check (pattern plus "ask the human"), saving about 5 hours across A14 and B10
2. Shims: Claude Code hooks only, dropping universal shims for the demo (keep them in the README as "next")
3. Test diff facts: judge-only, no deterministic facts (B11)
4. Panel: CAP DETECTED view only, no live feed

Never cut: the data destruction check, the human intent pop-up, and the CAP DETECTED screen. Those three are the project.

11. Demo script (3 minutes)

The demo is one real AI agent making three real mistakes, and nocap catching each one live. VS Code on the big screen: Claude Code in the terminal on the left, the nocap panel on the right. Font size up. Run `pnpm demo:reset` right before judges arrive.

Time	Say	Show
0:00–0:25	"AI agents now run commands on real systems. When instructions are vague, they guess, and a recent study found 56–68% of those runs break a boundary. Tools like dcg block known-bad commands. But they can't tell 3 test rows from 48,000 real users. nocap tests the command before it runs."	Panel empty, status bar says "nocap: on"
0:25–1:10	"I ask the agent to clean up our test users."	Type the task. Task card fills in. Agent writes the broad <code>DELETE</code> . CAP DETECTED: 48,213 rows, 3 admins, 2 cascaded tables. Task ✓, Agent says ✓, Actually does ✗.
1:10–1:25	"The agent fixes itself. And since I'd normally just hit yes, nocap makes me say why too."	Agent narrows to <code>@test.local</code> . Pop-up asks what I expect. I type "ok": refused. I type "delete the QA test accounts": matches 178 rows, green check, runs.
1:25–1:55	"Next: add embeddings for new documents."	Agent runs the backfill. CAP DETECTED: about \$340 against a \$20 budget. Agent adds the filter, estimate drops under \$1, runs.
1:55–2:30	"Last one. Every developer has seen an agent do this."	Task: fix the failing cart test. Agent edits the test's expected value. CAP DETECTED: the task says fix the code, the agent changed the test. Agent fixes <code>cart.ts</code> , test passes.

Time	Say	Show
2:30–2:50	"It works with any CLI agent through shims, and deeper with Claude Code through hooks. Safe commands pass in under 50 ms. Our judge scores X out of 30 on our eval set."	Stats header: 3 caught. Quick cut to the eval score in the README.
2:50–3:00	"nocap. Your agent can't cap anymore."	Landing page with the domain and install button

Backup plan: if the live agent does something unexpected, say "let's look at a recorded run" and play C16's video from the matching timestamp. Never debug in front of judges.

12. Risks and mitigations

The biggest risk is the live AI agent behaving differently during the demo, not the code. Every risk below has an owner and a fallback.

Risk	Likelihood	Owner	Mitigation
Claude Code doesn't make the mistake on cue, or fixes it differently	High	A	Tune prompts in A18 until 5/5; keep the backup video; if needed, give the agent a pre-written first command ("run this cleanup query")
Hackathon WiFi slow or down, Gemini calls time out	High	A	"Rules only" fallback (A15); phone hotspot; cache warm-up run right before judging
Dry run leaves a change in the database	Low, but serious	B	Rollback in <code>finally</code> ; the 100-run test in B8; <code>demo:reset</code> before every run
Claude Code's hook format differs from what we expect	Medium	A	A0.2 captures real payloads in hour 1, before any code is written
Shims not active because terminals opened before install	Medium	C	The relaunch-terminals button (C3); demo uses the Claude Code hook path anyway

Risk	Likelihood	Owner	Mitigation
MongoDB Atlas blocked on event WiFi	Medium	B	Local <code>.jsonl</code> fallback (B21); logging never blocks a verdict
Judge gives a wrong verdict on stage	Medium	A	Eval set (A17); the demo cases are in the eval set; policy rules back the judge up
Integration hell on Saturday night	Medium	All	Contracts locked in hour 1; stub daemon by hour 3; hour-18 gate
A judge says "dgc already does this"	High	C	Say it first in the pitch; show the numbers dgc can't produce
Team runs out of time	Medium	All	Cut list in section 10; feature freeze at hour 26

13. Stretch goals, judge Q&A, open decisions

Only start a stretch goal after the hour-26 feature freeze gates are all green. In priority order:

1. The cut items from section 10, restored in reverse order
2. The commit shim and native hook adapters for Codex CLI and Gemini CLI
3. Team feed: blocked actions posted to a shared channel so the whole team sees them
4. Deep check for production systems: parse `terraform plan` output and count resources destroyed
5. MCP server version of nocap, so any MCP-capable agent can call it

Judge questions to prepare

Question	Answer
How is this different from dgc or Claude Code's permission prompts?	They match patterns or ask yes/no. nocap measures the real effect and compares it with the task and the agent's stated intent. They can't tell you "48,213 rows"; we can.

Question	Answer
Isn't running the query dangerous?	It runs inside a transaction that always rolls back, with 3 s and 1 s timeouts. In production you'd point it at a replica. We say this in the README.
Can't the agent just lie about its intent?	Yes, and that's the point: the effect is measured, not taken on trust. A lie shows up as a mismatch.
What about an agent actively trying to get around nocap?	Out of scope, like dcg. We guard against well-meaning mistakes, which is what actually happens. Hooks plus shims make accidental bypass unlikely.
Does this slow the agent down?	Safe commands: under 50 ms. Only risky ones get the deep check, under 5 s.
How accurate is the AI judge?	Show the eval score (A17), and explain that hard rules back it up and never depend on the model.
Does my data leave my machine?	Only the Gemini call, with sensitive columns and secrets stripped first (FR-G7).
Why would anyone install this?	Anyone running agents with real database or API access. One install, one rules file, and it covers every agent.

Open decisions (settle Friday night)

- ☐ Final name and domain: nocap, or something else if the name is taken on npm and the Marketplace
- ☐ Which judge model tier in Gemini: the fastest one for latency, or a stronger one for accuracy (decide from A17's eval)
- ☐ Preact or plain TypeScript for the panel (C's call)
- ☐ Who presents (one speaker, one person driving the laptop)

Sources

- [Destructive Command Guard \(dgc\)](#): the main existing tool, pattern-based blocking across many agents
- [BUER](#): watches what agents do to code, including test tampering; does not infer intent
- [Hugging Face papers: agent underspecification study](#): 55.8–67.8% of runs violated at least one boundary
- [Claude Code hooks documentation](#): blocking dangerous operations before they execute